

Artificial Neural Network (ANN) (2)

Dr. Mohammed Brahimi and Dr. Aicha Boutorh



- 1 Standard Approach: Backpropagation and Gradient Descent
 - Backpropagation: The General Structure of the Algorithm
 - Backpropagation: Backpropagating the Error Gradients
 - Backpropagation: Updating the Weights in a Network
 - Backpropagation: The Algorithm
 - A Worked Example: Using Backpropagation to Train a Feedforward Network for a Regression Task



Standard Approach: Backpropagation and Gradient Descent

Backpropagation: The General Structure of the Algorithm

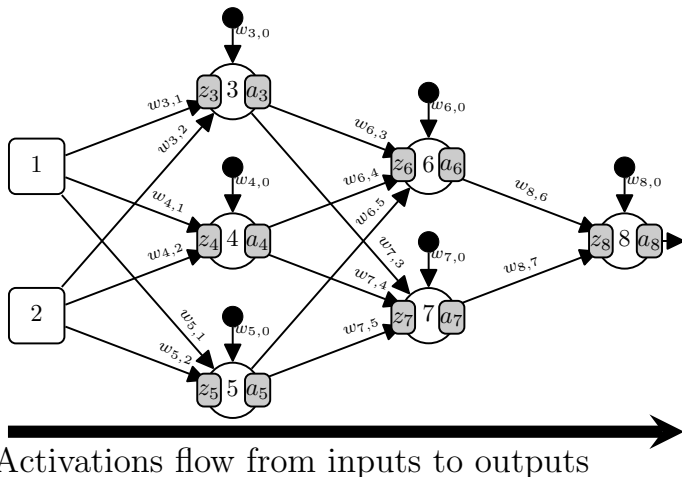


Figure 1: The calculation of the z values and activations of each neuron during the forward pass of the backpropagation algorithm. This figure is based on Figure 6.5 of (Kelleher, 2019).

Backpropagation: The General Structure of the Algorithm

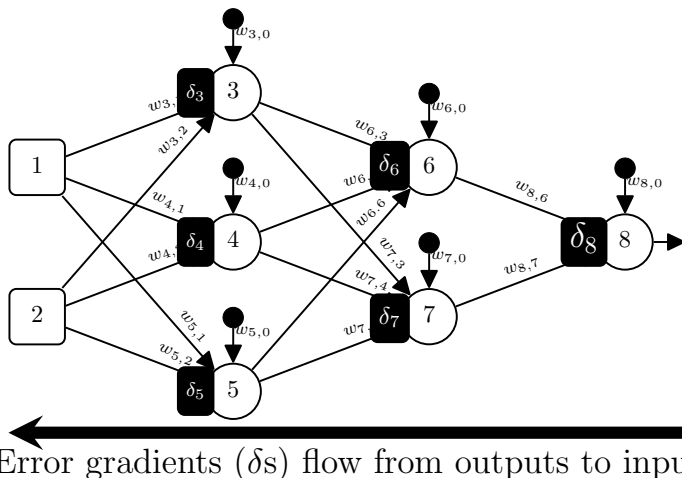


Figure 2: The backpropagation of the δ values during the backward pass of the backpropagation algorithm. This figure is based on Figure 6.6 of (Kelleher, 2019).

$$\delta_k = \frac{\partial \mathcal{E}}{\partial z_k} \quad (1)$$

$$\delta_k = \frac{\partial a_k}{\partial z_k} \times \frac{\partial \mathcal{E}}{\partial a_k} \quad (2)$$

$$\frac{d}{dz} \text{logistic}(z) = \text{logistic}(z) \times (1 - \text{logistic}(z)) \quad (3)$$

$$\begin{aligned}\frac{d}{dz} \text{logistic}(z=0) &= \text{logistic}(0) \times (1 - \text{logistic}(0)) \\ &= 0.5 \times (1 - 0.5) \\ &= 0.25\end{aligned}\tag{4}$$

1 1 3 1 1 3 3

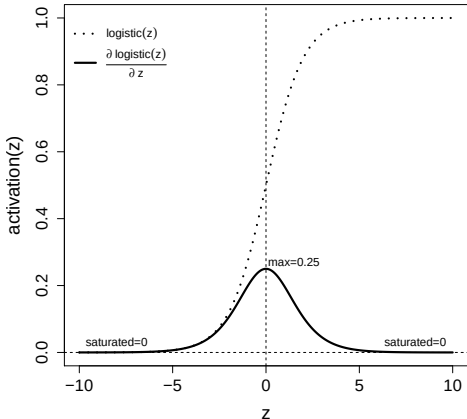


Figure 3: Plots of the logistic function and its derivative. This figure is Figure 4.6 of (Kelleher, 2019) and is used here with permission.

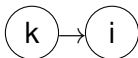
$$L_2(\mathbb{M}_{\mathbf{w}}, \mathcal{D}) = \frac{1}{2} \sum_{i=1}^n (t_i - \mathbb{M}_{\mathbf{w}}(\mathbf{d}_i))^2 \quad (5)$$

$$\frac{\partial}{\partial \mathbf{w}[j]} L_2(\mathbb{M}_{\mathbf{w}}, \mathbf{d}) = (t - \mathbb{M}_{\mathbf{w}}(\mathbf{d})) \times -\mathbf{d}[j] \quad (6)$$

$$\frac{\partial \mathcal{E}}{\partial a_k} = \frac{\partial L_2(\mathbb{M}_{\mathbf{w}}, \mathbf{d})}{\partial \mathbb{M}_{\mathbf{w}}(\mathbf{d})} = t - \mathbb{M}_{\mathbf{w}}(\mathbf{d}) = t_k - a_k \quad (7)$$

$$\frac{\partial \mathcal{E}}{\partial a_k} = -(t_k - a_k) \quad (8)$$

$$=(\text{logistic}(z) \times (1 - \text{logistic}(z))) \times -(t_k - a_k) \quad (9)$$



$$\frac{\partial \mathcal{E}}{\partial a_k} = \sum_{i=1}^n w_{i,k} \times \delta_i \quad (10)$$

$$\begin{aligned}
 \delta_k &= \frac{\partial a_k}{\partial z_k} \times \frac{\partial \mathcal{E}}{\partial a_k} \\
 &= \frac{\partial a_k}{\partial z_k} \times \left(\sum_{i=1}^n w_{i,k} \times \delta_i \right) \\
 &= \underbrace{\frac{d}{dz} \text{logistic}(z)}_{\text{Assuming a logistic activation function}} \times \left(\sum_{i=1}^n w_{i,k} \times \delta_i \right) \\
 &= \underbrace{(\text{logistic}(z) \times (1 - \text{logistic}(z)))}_{\text{Assuming a logistic activation function}} \times \left(\sum_{i=1}^n w_{i,k} \times \delta_i \right) \quad (11)
 \end{aligned}$$

$$\frac{\partial \mathcal{E}}{\partial w_{i,k}} = \frac{\partial \mathcal{E}}{\partial a_i} \times \frac{\partial a_i}{\partial z_i} \times \frac{\partial z_i}{\partial w_{i,k}} \quad (12)$$

$$\begin{aligned} \frac{\partial \mathcal{E}}{\partial w_{i,k}} &= \frac{\partial \mathcal{E}}{\partial a_i} \times \frac{\partial a_i}{\partial z_i} \times \frac{\partial z_i}{\partial w_{i,k}} \\ &= \delta_i \times \frac{\partial z_i}{\partial w_{i,k}} \end{aligned} \quad (13)$$

$$\frac{\partial z_i}{\partial w_{i,k}} = a_k \quad (14)$$

$$\begin{aligned}\frac{\partial \mathcal{E}}{\partial w_{i,k}} &= \frac{\partial \mathcal{E}}{\partial a_i} \times \frac{\partial a_i}{\partial z_i} \times \frac{\partial z_i}{\partial w_{i,k}} \\ &= \delta_i \times \frac{\partial z_i}{\partial w_{i,k}} \\ &= \delta_i \times a_k\end{aligned}\tag{15}$$



Backpropagation: Updating the Weights in a Network

$$w_{i,k} \leftarrow w_{i,k} - \alpha \times \delta_i \times a_k \quad (16)$$

$$\Delta w_{i,k} = \sum_{j=1}^m \delta_{i,j} \times a_{k,j} \quad (17)$$

$$w_{i,k} \leftarrow w_{i,k} - \alpha \times \Delta w_{i,k} \quad (18)$$



Backpropagation: The Algorithm

Require: set of training instances $\mathcal{D}$

Require: a learning rate α that controls how quickly the algorithm converges

Require: a batch size B specifying the number of examples in each batch

Require: a convergence criterion


```

1: Shuffle  $\mathcal{D}$  and create the mini-batches:  $[(\mathbf{X}^{(1)}, \mathbf{Y}^{(1)}), \dots, (\mathbf{X}^k, \mathbf{Y}^k)]$ 
2: Initialize the weight matrices for each layer:  $\mathbf{W}^{(1)}, \dots, \mathbf{W}^{(L)}$ 
3: repeat ▷ Each repeat loop is one epoch
4:   for  $t=1$  to number of mini-batches do ▷ Each for loop is one iteration
5:      $\mathbf{A}^{(0)} \leftarrow \mathbf{X}^{(t)}$ 
6:     for  $l=1$  to  $L$  do
7:        $\mathbf{v} \leftarrow [1_0, \dots, 1_m]$  ▷ Create  $\mathbf{v}$  the vector of bias terms
8:        $\mathbf{A}^{(l-1)} \leftarrow [\mathbf{v}; \mathbf{A}^{(l-1)}]$  ▷ Insert  $\mathbf{v}$  into the activation matrix
9:        $\mathbf{Z}^{(l)} \leftarrow \mathbf{W}^l \mathbf{A}^{(l-1)}$ 
10:       $\mathbf{A}^{(l)} \leftarrow \varphi(\mathbf{Z}^{(l)})$  ▷ Elementwise application of  $\varphi$  to  $\mathbf{Z}^{(l)}$ 
11:    end for
12:    for each weight  $w_{i,k}$  in the network do
13:       $\Delta w_{i,k} = 0$ 
14:    end for
15:    for each example in the mini-batch do ▷ Backpropagate the  $\delta$ s
16:      for each neuron  $i$  in the output layer do
17:         $\delta_i = \frac{\partial \mathcal{E}}{\partial a_i} \times \frac{\partial a_i}{\partial z_i}$  ▷ See Equation (9)[10]
18:      end for
19:      for  $l = L-1$  to  $1$  do
20:        for each neuron  $i$  in the layer  $l$  do
21:           $\delta_i = \frac{\partial \mathcal{E}}{\partial a_i} \times \frac{\partial a_i}{\partial z_i}$  ▷ See Equation (11)[11]
22:        end for
23:      end for
24:      for each weight  $w_{i,k}$  in the network do
25:         $\Delta w_{i,k} = \Delta w_{i,k} + (\delta_i \times a_k)$  ▷ Equation (17)[15]
26:      end for
27:    end for
28:    for each weight  $w_{i,k}$  in the network do
29:       $w_{i,k} \leftarrow w_{i,k} - \alpha \times \Delta w_{i,k}$  ▷ Equation (18)[15]
30:    end for
31:  end for
32:  shuffle  $[(\mathbf{X}^{(1)}, \mathbf{Y}^{(1)}), \dots, (\mathbf{X}^k, \mathbf{Y}^k)]$ 
33: until convergence occurs

```

Table 1: Hourly samples of ambient factors and full load electrical power output of a combined cycle power plant.

ID	AMBIENT TEMPERATURE °C	RELATIVE HUMIDITY %	ELECTRICAL OUTPUT MW
1	03.21	86.34	491.35
2	31.41	68.50	430.37
3	19.31	30.59	463.00
4	20.64	99.97	447.14

Table 2: The minimum and maximum values for the AMBIENT TEMPERATURE, RELATIVE HUMIDITY, and ELECTRICAL OUTPUT features in the power plant dataset.

	AMBIENT TEMPERATURE	RELATIVE HUMIDITY	ELECTRICAL OUTPUT
Min	1.81°C	25.56%	420.26MW
Max	37.11°C	100.16%	495.76MW

Table 3: The *range-normalized* hourly samples of ambient factors and full load electrical power output of a combined cycle power plant, rounded to two decimal places.

ID	AMBIENT TEMPERATURE °C	RELATIVE HUMIDITY %	ELECTRICAL OUTPUT MW
1	0.04	0.81	0.94
2	0.84	0.58	0.13
3	0.50	0.07	0.57
4	0.53	1.00	0.36

A Worked Example: Using Backpropagation to Train a Feedforward Network for a Regression Task

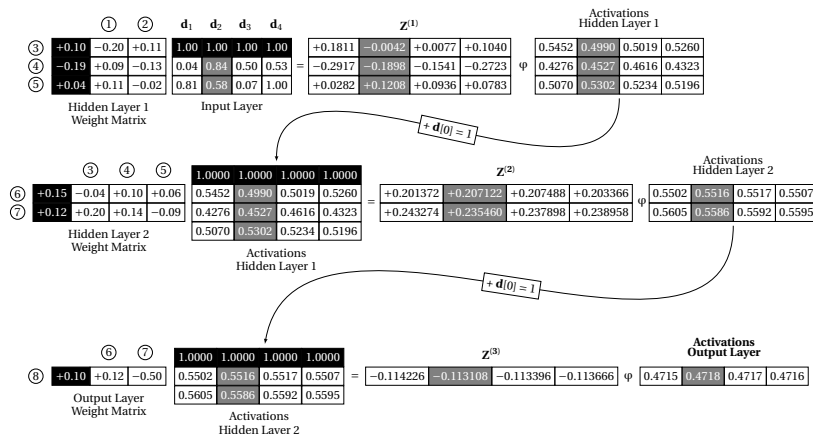


Figure 4: The forward pass of the examples listed in Table 3^[20] through the network in Figure ??^[??].

Table 4: The per example error after the forward pass illustrated in Figure 4^[21], the per example $\partial\mathcal{E}/\partial a_8$, and the **sum of squared errors** for the model over the dataset of four examples.

	d_1	d_2	d_3	d_4
Target	0.9400	0.1300	0.5700	0.3600
Prediction	0.4715	0.4718	0.4717	0.4716
Error	0.4685	-0.3418	0.0983	-0.1116
$\partial\mathcal{E}/\partial a_8$: Error $\times -1$	-0.4685	0.3418	-0.0983	0.1116
Error ²	0.21949225	0.11682724	0.00966289	0.01245456
SSE:				0.17921847

A Worked Example: Using Backpropagation to Train a Feedforward Network for a Regression Task

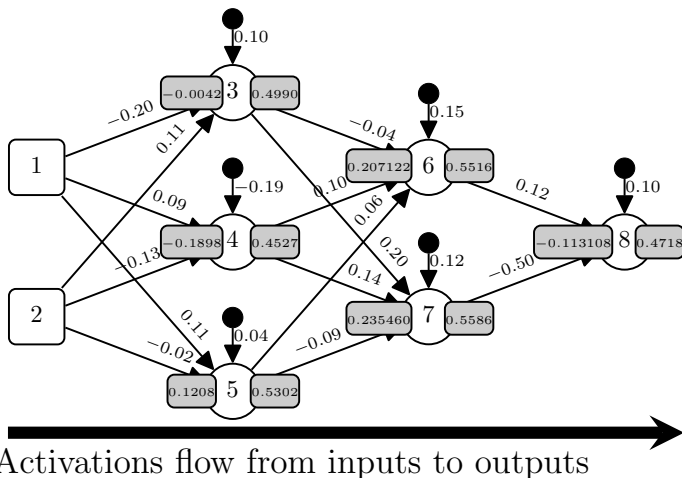


Figure 5: An illustration of the forward propagation of d_2 through the network showing the weights on each connection, and the weighted sum z and activation a value for each neuron in the network.

$$\frac{\partial \mathcal{E}}{\partial a} = 0.3418 \quad (19)$$

$$\begin{aligned} \delta_8 &= \frac{\partial \mathcal{E}}{\partial a_8} \times \frac{\partial a_8}{\partial z_8} \\ &= 0.3418 \times 0.2492 \\ &= 0.0852 \end{aligned} \quad (20)$$

Table 5: The $\partial a / \partial z$ for each neuron for Example 2 rounded to four decimal places.

NEURON	z	$\partial a / \partial z$
3	-0.004200	0.2500
4	-0.189800	0.2478
5	0.120800	0.2491
6	0.207122	0.2473
7	0.235460	0.2466
8	-0.113108	0.2492

A Worked Example: Using Backpropagation to Train a Feedforward Network for a Regression Task

$$\begin{aligned}
 \delta_6 &= \frac{\partial \mathcal{E}}{\partial a_6} \times \frac{\partial a_6}{\partial z_6} \\
 &= \left(\sum \delta_i \times w_{i,6} \right) \times \frac{\partial a_6}{\partial z_6} \\
 &= (\delta_8 \times w_{8,6}) \times \frac{\partial a_6}{\partial z_6} \\
 &= (0.0852 \times 0.12) \times 0.2473 \\
 &= 0.0025
 \end{aligned} \tag{21}$$

$$\begin{aligned}
 \delta_7 &= \frac{\partial \mathcal{E}}{\partial a_7} \times \frac{\partial a_7}{\partial z_7} \\
 &= \left(\sum \delta_i \times w_{i,7} \right) \times \frac{\partial a_7}{\partial z_7} \\
 &= (\delta_8 \times w_{8,7}) \times \frac{\partial a_6}{\partial z_6} \\
 &= (0.0852 \times -0.50) \times 0.2466 \\
 &= -0.0105
 \end{aligned} \tag{22}$$

A Worked Example: Using Backpropagation to Train a Feedforward Network for a Regression Task

$$\begin{aligned}
 \delta_3 &= \frac{\partial \mathcal{E}}{\partial a_3} \times \frac{\partial a_3}{\partial z_3} \\
 &= \left(\sum \delta_i \times w_{i,3} \right) \times \frac{\partial a_3}{\partial z_3} \\
 &= ((\delta_6 \times w_{6,3}) + (\delta_7 \times w_{7,3})) \times \frac{\partial a_3}{\partial z_3} \\
 &= ((0.0025 \times -0.04) + (-0.0105 \times 0.20)) \times 0.2500 \\
 &= -0.0006 \quad (23)
 \end{aligned}$$

A Worked Example: Using Backpropagation to Train a Feedforward Network for a Regression Task

$$\begin{aligned}
 \delta_4 &= \frac{\partial \mathcal{E}}{\partial a_4} \times \frac{\partial a_4}{\partial z_4} \\
 &= \left(\sum \delta_i \times w_{i,4} \right) \times \frac{\partial a_4}{\partial z_4} \\
 &= ((\delta_6 \times w_{6,4}) + (\delta_7 \times w_{7,4})) \times \frac{\partial a_4}{\partial z_4} \\
 &= ((0.0025 \times 0.10) + (-0.0105 \times 0.14)) \times 0.2478 \\
 &= -0.0003
 \end{aligned} \tag{24}$$

A Worked Example: Using Backpropagation to Train a Feedforward Network for a Regression Task

$$\begin{aligned}
 \delta_5 &= \frac{\partial \mathcal{E}}{\partial a_5} \times \frac{\partial a_5}{\partial z_5} \\
 &= \left(\sum \delta_i \times w_{i,5} \right) \times \frac{\partial a_5}{\partial z_5} \\
 &= ((\delta_6 \times w_{6,5}) + (\delta_7 \times w_{7,5})) \times \frac{\partial a_5}{\partial z_5} \\
 &= ((0.0025 \times 0.06) + (-0.0105 \times -0.09)) \times 0.2491 \\
 &= 0.0003 \qquad (25)
 \end{aligned}$$

A Worked Example: Using Backpropagation to Train a Feedforward Network for a Regression Task

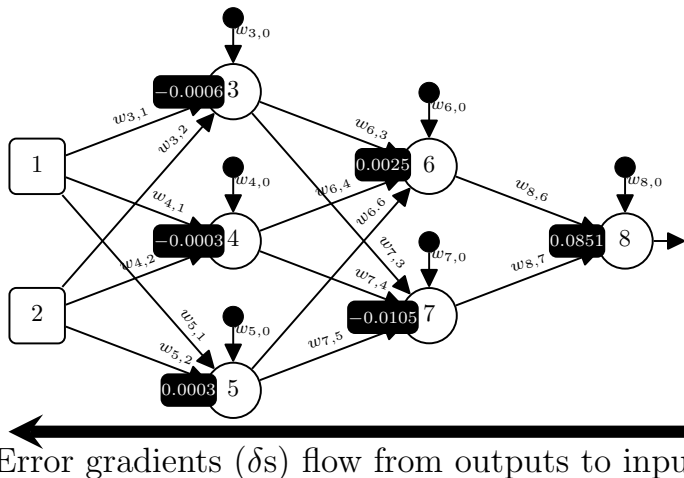


Figure 6: The δ s for each of the neurons in the network for Ex. 2.

Table 6: The $\partial\mathcal{E}/\partial w_{i,k}$ calculations for d_2 for every weight in the network. The neuron index 0 denotes the bias input for each neuron.

NEURON _{<i>i</i>}	NEURON _{<i>k</i>}	$w_{i,k}$	δ_i	a_k	$\partial\mathcal{E}/\partial w_{i,k}$
8	0	$w_{8,0}$	0.0852	1	$0.0852 \times 1 = 0.0852$
8	6	$w_{8,6}$	0.0852	0.5516	$0.0852 \times 0.5516 = 0.04699632$
8	7	$w_{8,7}$	0.0852	0.5586	$0.0852 \times 0.5586 = 0.04759272$
7	0	$w_{7,0}$	-0.0105	1	$-0.0105 \times 1 = -0.0105$
7	3	$w_{7,3}$	-0.0105	0.4990	$-0.0105 \times 0.4527 = -0.0052395$
7	4	$w_{7,4}$	-0.0105	0.4527	$-0.0105 \times 0.4527 = -0.00475335$
7	5	$w_{7,5}$	-0.0105	0.5302	$-0.0105 \times 0.5302 = -0.0055671$
6	0	$w_{6,0}$	0.0025	1	$0.0025 \times 1 = 0.0025$
6	3	$w_{6,3}$	0.0025	0.4990	$0.0025 \times 0.4527 = 0.0012475$
6	4	$w_{6,4}$	0.0025	0.4527	$0.0025 \times 0.4527 = 0.00113175$
6	5	$w_{6,5}$	0.0025	0.5302	$0.0025 \times 0.5302 = 0.0013255$
5	0	$w_{5,0}$	0.0003	1	$0.0003 \times 1 = 0.0003$
5	1	$w_{5,1}$	0.0003	0.84	$0.0003 \times 0.84 = 0.000252$
5	2	$w_{5,2}$	0.0003	0.58	$0.0003 \times 0.58 = 0.000174$
4	0	$w_{4,0}$	-0.0003	1	$-0.0003 \times 1 = -0.0003$
4	1	$w_{4,1}$	-0.0003	0.84	$-0.0003 \times 0.84 = -0.000252$
4	2	$w_{4,2}$	-0.0003	0.58	$-0.0003 \times 0.58 = -0.000174$
3	0	$w_{3,0}$	-0.0006	1	$-0.0006 \times 1 = -0.0006$
3	1	$w_{3,1}$	-0.0006	0.84	$-0.0006 \times 0.84 = -0.000504$
3	2	$w_{3,2}$	-0.0006	0.58	$-0.0006 \times 0.58 = -0.000348$

A Worked Example: Using Backpropagation to Train a Feedforward Network for a Regression Task

$$\begin{aligned}
 w_{7,5} &= w_{7,5} - \alpha \times \delta_7 \times a_5 \\
 &= w_{7,5} - \alpha \times \frac{\partial \mathcal{E}}{\partial w_{i,k}} \\
 &= -0.09 - 0.2 \times -0.0055671 \\
 &= -0.08888658
 \end{aligned}
 \tag{26}$$

Table 7: The calculation of $\Delta w_{7,5}$ across our four examples.

MINI-BATCH EXAMPLE	$\frac{\partial \mathcal{E}}{\partial w_{7,5}}$
$\mathbf{d}_1$	0.00730080
$\mathbf{d}_2$	-0.00556710
$\mathbf{d}_3$	0.00157020
$\mathbf{d}_4$	-0.00176664
$\Delta w_{7,5} =$	0.00153726

$$\begin{aligned}w_{7,5} &= w_{7,5} - \alpha \times \Delta w_{i,k} \\&= -0.09 - 0.2 \times 0.00153726 \\&= -0.0903074520\end{aligned}\tag{27}$$

Table 8: The per example error after each weight has been updated once, the per example $\partial\mathcal{E}/\partial a_8$, and the **sum of squared errors** for the model.

	d_1	d_2	d_3	d_4
Target	0.9400	0.1300	0.5700	0.3600
Prediction	0.4738	0.4741	0.4740	0.4739
Error	0.4662	-0.3441	0.0960	-0.1139
$\partial\mathcal{E}/\partial a_8$: Error $\times -1$	-0.4662	0.3441	-0.0960	0.1139
Error ²	0.21734244	0.11840481	0.009216	0.01297321
SSE:				0.17896823

Table 9: The per example prediction, error, and the sum of squared errors after training has converged to an $SSE < 0.0001$.

	d_1	d_2	d_2	d_2
Target	0.9400	0.1300	0.5700	0.3600
Prediction	0.9266	0.1342	0.5700	0.3608
Error	0.0134	-0.0042	0.0000	-0.0008
Error ²	0.00017956	0.00001764	0.00000000	0.00000064
SSE:				0.00009892

A Worked Example: Using Backpropagation to Train a Feedforward Network for a Regression Task

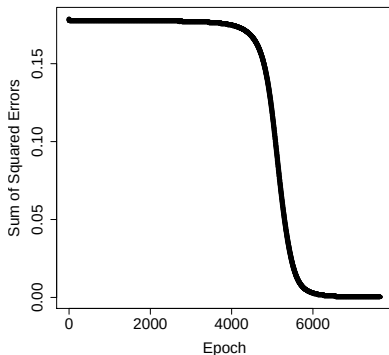


Figure 7: A plot showing how the sum of squared errors of the network changed during training.

1 Standard Approach: Backpropagation and Gradient Descent

Slide Acknowledgment

The slides used in this course are based on the official textbook materials, with modifications made where necessary to suit the course requirements and enhance the learning experience.

Kelleher, John D. 2019. *Deep learning. Essential knowledge series*. MIT Press.